2111540642 Nuhash Bin Hassan | 2232755642 Abu Md. Ashraful Rahat | 2311086642 Md. Shakib Rafid Shakib | 2321534042 Muhammad Farhan Rafid

AI-POWERED E-COMMERCE PLATFORM WITH INTELLIGENT BUSINESS INSIGHTS

WEEKLY PROJECT UPDATE REPORT 3 - CLASS 6

Abstract - This update expanded the storefront and backend structure and changed the customer chatbot from a general conversational tool into a product-aware assistant. The work combined Shakib's broader React interface, Rahat's modular Django REST API foundation and Farhan's database-connected Ollama chatbot, followed by direct API and browser testing.

1. SHAKIB - FRONTEND PROGRESS

Shakib organized React/Vite pages for the home screen, registration, login, profile, product listing and details, category listing and details, cart, checkout, order history, order details, AI chat and invalid routes. React Router connected these screens, and the reusable navigation bar remained available throughout the application.

Basic shopping behavior was represented through React state. Users could add products to a cart, adjust quantities, remove items, proceed to checkout and view order-related pages. The chatbot continued to send JSON messages to /api/ai/chat/ and render the backend reply.

2. RAHAT - BACKEND AND REST API

Rahat structured the Django backend into accounts, store, cart, orders, chat and core applications. Models covered user profiles, categories, products, product images, inventory records, carts, cart items, orders, payments, invoices, chat sessions, chat messages, human-support sessions, settings and notifications.

Serializers, Django REST Framework ViewSets and router-based URLs were prepared for these records. JWT login and refresh endpoints, CORS support and permission classes were configured so the React frontend could communicate with protected backend resources. Docker files and backend documentation supported repeatable setup.

3. FARHAN - PRODUCT-AWARE CHATBOT

Farhan maintained a separate Django chatbot application connected to the local Ollama phi3:mini model. The /api/ai/chat/ endpoint accepted customer questions and returned generated responses to the React interface.

The chatbot view was extended to query the Django Product model before calling Ollama. It retrieved active products that were in stock and supported listing available products, searching by product name, checking prices and descriptions, and answering simple price-filter questions. This reduced invented product information and made replies depend on store records.

4. DEMONSTRATION DATA AND TESTING

An Electronics category and five sample products were entered through Django Admin: Wireless Mouse, Laptop Stand, USB-C Charger, Gaming Headset and Mechanical Keyboard. Tests covered listing products, filtering items under Tk 2000 and requesting details about the Wireless Mouse.

The feature was checked through direct PowerShell API requests and the React chatbot page. Django system checking completed without reported issues, and the working change was committed to the FarhanRafid branch.

5. CURRENT PROJECT STATUS

The project contained a functional storefront prototype, a modular Django backend with REST API definitions and a customer chatbot connected to actual product records. The demonstrated flow was React frontend -> Django chatbot API -> Product database and Ollama -> React reply.

The frontend, backend and chatbot branches had not yet been fully merged. Some pages still relied on local React state, and Rahat's full API set required systematic browser and Postman testing. SQLite demonstration records also did not transfer automatically through Git.

6. NEXT DEVELOPMENT TASKS

The next phase was planned to combine the latest frontend and backend versions, resolve migrations and connect product, cart, checkout, account and order pages to the REST APIs. A reusable fixture or seed command was also needed so every member could load the same products.

Additional work included a formal Postman collection, stronger authentication and permission testing, improved error handling, README updates, a one-minute demonstration video, the admin analytics chatbot and the planned human-support takeover feature.

7. FILES AND COMPONENTS UPDATED

Main work involved the Django chatbot view and URL endpoint, the React chat page, product database records and supporting project configuration. No existing Rahat or Shakib application was intentionally removed during this update.

REFERENCES AND TOOLS

[1] Django 4.2 documentation. [2] Django REST Framework ViewSets, routers and permissions. [3] SimpleJWT documentation. [4] React, React Router and Vite documentation. [5] Ollama Python library. [6] Project GitHub history. [7] OpenAI ChatGPT used for explanation, debugging and report preparation; features were locally tested by the team.